UNIVERSIDAD NACIONAL DE SAN AGUSTÍN AREQUIPA

FACULTAD DE INGENIERÍA DE PRODUCCIÓN DE SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN

Laboratorio de Algoritmos de Búsqueda

Curso:
Tópicos en Inteligencia Artificial

Alumnos:
Ferro Vásquez, Arleen Maritza
Huanca Olazabal, Cristhian David
Mamani Casilla, Wilson Isaac

Docente:
Juan Carlos Gutierrez Cáceres

Semestre:
IX

Arequipa - Perú
2026

Algortimos de Búsqueda

16 de mayo de 2026

1. Introducción

El problema de encontrar el camino más corto entre dos puntos dentro de un gráfico o red es un pilar fundamental en las ciencias de la computación, con aplicaciones directas en la robótica, los videojuegos, los sistemas de información geográfica (SIG) y el enrutamiento de paquetes de datos.

Este proyecto implementa un software interactivo en C++ que genera mapas matriciales con obstáculos aleatorios y permite evaluar en tiempo real el comportamiento, el costo computacional y la eficiencia espacial de múltiples estrategias de búsqueda. Gracias al uso de la biblioteca de visión artificial **OpenCV**, el usuario puede seleccionar nodos de origen y destino dinámicamente mediante clics en pantalla y alternar entre diversos solucionadores para comparar visualmente las trayectorias resultantes y los nodos explorados.

2. Arquitectura del Sistema

El desarrollo del software se estructuró bajo un modelo modular, dividiendo las responsabilidades en tres capa lógicas esenciales representadas en sus respectivos archivos de cabecera e implementación:

- `grid.h` **(Modelo del Entorno):** Define la abstracción del plano espacial mediante una estructura lineal indexada que emula una matriz de ocupación bidimensional. Maneja la validez geométrica de los movimientos y la aleatoriedad de los obstáculos.

```cpp
    class GridMap {
public:
    int cols, rows;
    std::vector<bool> active;
    GridMap(int c, int r) : cols(c), rows(r), active(c * r, true) {}

    void randomize(float chance) {
        srand((unsigned)time(nullptr));
        for (int i = 0; i < cols * rows; ++i) active[i] = ((float)rand() / RAND_MAX) > chance;
    }

    inline int toIdx(int c, int r) const { return r * cols + c; }
    inline int toCol(int i) const { return i % cols; }
    inline int toRow(int i) const { return i / cols; }
    bool isWalkable(int c, int r) const {
        return c >= 0 && c < cols && r >= 0 && r < rows && active[toIdx(c, r)];
    }
};

```

Listing 1: Lógica de grid.h

- `solvers.h` **(Capa Algorítmica):** Contiene la lógica matemática y estructural de las estrategias de búsqueda de caminos. Centraliza los datos del resultado en una estructura compacta común (`SearchResult`).

```cpp
    enum SearchMode { ASTAR, DIJKSTRA, BFS, DFS };

struct SearchResult {
    std::vector<int> path;
    std::vector<int> explored;
    float cost = 0.0f;
    bool found = false;
```

```
    SearchMode modeUsed;
};

class PathFinder {
public:
    SearchResult solve(const GridMap& map, int start, int end, SearchMode mode) {
        int N = map.cols * map.rows;
        std::vector<float> g(N, 1e9);
        std::vector<int> parent(N, -1);
        std::vector<bool> closed(N, false);
        SearchResult res;
        res.modeUsed = mode;

        auto h = [&](int a, int b) {
            if (mode != ASTAR) return 0.0f;
            int dx = abs(map.toCol(a) - map.toCol(b)), dy = abs(map.toRow(a) - map.toRow(b));
            return (dx + dy) + (1.4142f - 2.0f) * std::min(dx, dy);
        };

        std::priority_queue<std::pair<float, int>, std::vector<std::pair<float, int>>, std::greater<>> openPQ;
        std::queue<int> openQ;
        std::stack<int> openS;

        g[start] = 0;
        if (mode == ASTAR || mode == DIJKSTRA) openPQ.push({ h(start, end), start });
        else if (mode == BFS) openQ.push(start);
        else openS.push(start);

        int dc[8] = {1, 0, -1, 0, 1, 1, -1, -1}, dr[8] = {0, 1, 0, -1, 1, -1, 1, -1};
        float stepCosts[8] = {1, 1, 1, 1, 1.4142f, 1.4142f, 1.4142f, 1.4142f};

        while (true) {
            int curr;
            if (mode == ASTAR || mode == DIJKSTRA) {
                if (openPQ.empty()) break;
                curr = openPQ.top().second; openPQ.pop();
            } else if (mode == BFS) {
                if (openQ.empty()) break;
                curr = openQ.front(); openQ.pop();
            } else {
                if (openS.empty()) break;
                curr = openS.top(); openS.pop();
            }

            if (closed[curr]) continue;
            closed[curr] = true;
            res.explored.push_back(curr);
            if (curr == end) { res.found = true; break; }

            for (int d = 0; d < 8; ++d) {
                int nc = map.toCol(curr) + dc[d], nr = map.toRow(curr) + dr[d];
                if (!map.isWalkable(nc, nr)) continue;
                int neighbor = map.toIdx(nc, nr);
                float newG = g[curr] + stepCosts[d];
                if (newG < g[neighbor]) {
                    g[neighbor] = newG;
```

```
                    parent[neighbor] = curr;
                    if (mode == ASTAR || mode == DIJKSTRA) openPQ.push({ newG + h(neighbor, end), neighbor });
                    else if (mode == BFS) openQ.push(neighbor);
                    else openS.push(neighbor);
                }
            }
        }
        if (res.found) {
            for (int v = end; v != -1; v = parent[v]) res.path.push_back(v);
            res.cost = g[end];
        }
        return res;
    }
};

```

Listing 2: Lógica de solvers.h

- `main.cpp` / `App` **(Controlador e Interfaz Gráfica):** Implementa el ciclo principal de la aplicación, el procesamiento de eventos de hardware (ratón y teclado), la renderización de primitivas geométricas en un *canvas* de imagen y la persistencia de datos.

```
    class App {
public:
    GridMap map;
    PathFinder finder;
    SearchResult result;
    SearchMode currentMode = ASTAR;
    int startNode = -1, endNode = -1;
    double lastSearchTime = 0.0;
    const float PAD = 30.0f;

    App() : map(GRID_COLS, GRID_ROWS) { map.randomize(0.35f); }

    void guardarNodosTxt() {
        if (result.explored.empty()) return;

        std::ofstream archivo("nodos_explorados.txt");
        if (archivo.is_open()) {
            for (int idx : result.explored) {
                archivo << map.toRow(idx) << ", " << map.toCol(idx) << "\n";
            }
            archivo.close();
            std::cout << "> Nodos explorados guardados en 'nodos_explorados.txt'" << std::endl;
        }
    }

    void runSearch() {
        if (startNode != -1 && endNode != -1) {
            auto start = std::chrono::high_resolution_clock::now();
            result = finder.solve(map, startNode, endNode, currentMode);
            auto end = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double, std::milli> duration = end - start;
            lastSearchTime = duration.count();
            guardarNodosTxt();
        }
    }

```

Listing 3: Lógica de la Clase App

3. Algoritmos de Búsqueda Implementados

El sistema permite comparar de forma inmediata cuatro algoritmos distribuidos en dos familias teóricas claras:

3.1. Algoritmos Basados en Costo e Informados

1. **Algoritmo A* (A-Estrella):** Es un método de búsqueda informado (heurístico). Utiliza la función de evaluación $f(n) = g(n) + h(n)$, donde $g(n)$ representa el costo acumulado real desde el inicio hasta el nodo actual n, y $h(n)$ es la estimación heurística hasta el objetivo. En esta aplicación se utiliza una **heurística diagonal u octil**, matemáticamente coherente con los 8 grados de libertad permitidos en la cuadrícula:

$$h(n) = (dx + dy) + (\sqrt{2} - 2) \times \text{mín}(dx, dy) \tag{1}$$

   Donde $dx = |x_n - x_{destino}|$ y $dy = |y_n - y_{destino}|$.

2. **Algoritmo de Dijkstra:** Variante clásica de búsqueda no informada que asume $h(n) = 0$. Explora los nodos radialmente en círculos concéntricos de costo incremental, garantizando la optimalidad del camino en grafos ponderados con pesos no negativos.

3.2. Algoritmos Basados en Estructuras de Datos Puras

1. **BFS (Búsqueda en Anchura):** Utiliza una estructura de cola (*First-In, First-Out - FIFO*). Explora por niveles ortogonales y diagonales completos. Dado que el mapa posee variaciones de costo en sus diagonales ($\sqrt{2} \approx 1{,}4142$), BFS encuentra el camino con el *menor número de aristas*, mas no necesariamente el camino de menor costo métrico.

2. **DFS (Búsqueda en Profundidad):** Emplea una estructura de pila (*Last-In, First-Out - LIFO*). Explora agresivamente una sola rama hasta el límite antes de realizar *backtracking*. No garantiza en absoluto rutas óptimas ni caminos de mínimo coste.

4. Resultados Experimentales y Análisis

A continuación, se presentan los resultados cuantitativos y cualitativos obtenidos de la ejecución interactiva de los cuatro algoritmos sobre el mismo escenario de prueba (cuadrícula de 50×50).

4.1. Tabla Comparativa de Métricas

Las métricas exactas extraídas directamente de las pruebas de rendimiento del simulador gráfico se consolidan en la Tabla 1.

Cuadro 1: Comparativa de Rendimiento Basada en los Ensayos del Simulador.

Algoritmo	Nodos Explorados	Costo Total	Tiempo de Ejecución
A*	37	15.0	0.172 ms
BFS	335	15.0	0.222 ms
Dijkstra	346	15.0	0.368 ms
DFS	371	387.0	0.309 ms

4.2. Análisis Crítico de los Comportamientos y Validación del Código

El comportamiento macroscópico observado en las capturas del simulador y las métricas recolectadas tienen su origen y validación directa en las estructuras de datos y condicionales lógicos implementados en la clase `PathFinder` del archivo `solvers.h`:

- **Eficiencia Espacial de A* (37 nodos explorados):** El drástico ahorro de exploración se valida matemáticamente en el código mediante la función lambda que calcula la distancia octil para los 8 grados de libertad:

```
    auto h = [&](int a, int b) {
        if (mode != ASTAR) return 0.0f;
        int dx = abs(map.toCol(a) - map.toCol(b)), dy = abs(map.toRow(a) - map.toRow(b));
        return (dx + dy) + (1.4142f - 2.0f) * std::min(dx, dy);
    };

```

Listing 4: Cálculo de la heurística Octil en solvers.h

Esta estimación se suma al costo real acumulado (g) al momento de insertar el nodo en la cola de prioridad:

```
    if (mode == ASTAR || mode == DIJKSTRA) openPQ.push({ newG + h(neighbor, end), neighbor });

```

Al estar ordenada de menor a mayor costo estimado, la estructura `std::priority_queue` fuerza al ciclo a extraer primero los nodos que se dirigen en línea recta u octil hacia el objetivo, reduciendo la exploración a una línea casi directa de solo 37 nodos.

- **Simetría y Optimalidad en Dijkstra frente a BFS (346 vs 335 nodos):** La diferencia en el volumen de exploración se valida en cómo gestiona cada algoritmo los pesos de las diagonales. Mientras que `BFS` opera ciegamente por orden de llegada en una cola FIFO común (`std::queue`):

```
    curr = openQ.front(); openQ.pop();

```

El algoritmo de `Dijkstra` analiza minuciosamente el peso real del desplazamiento utilizando el arreglo de costos ponderados que definiste en el código:

```
    int dc[8] = {1, 0, -1, 0, 1, 1, -1, -1}, dr[8] = {0, 1, 0, -1, 1, -1, 1, -1};
    float stepCosts[8] = {1, 1, 1, 1, 1.4142f, 1.4142f, 1.4142f, 1.4142f};

```

Listing 5: Ponderación de movimientos ortogonales y diagonales.

Al evaluar el costo real de las diagonales como $\sqrt{2} \approx 1{,}4142$, `Dijkstra` revalúa constantemente si el camino actual es mejor a través de la condicional `if (newG <g[neighbor])`, lo que genera un frente de expansión ligeramente más riguroso (346 nodos) para asegurar la precisión métrica absoluta.

- **La Ruta Errática de DFS (371 nodos explorados y Costo 387.0):** El comportamiento caótico y serpenteante que se visualiza en el canvas se valida por el uso de una estructura de Pila (`std::stack`), la cual procesa los nodos bajo la filosofía LIFO (*Last-In, First-Out*):

```
    } else {
        if (openS.empty()) break;
        curr = openS.top(); openS.pop();
    }

```

Listing 6: Extracción LIFO que genera el comportamiento de DFS

Al extraer siempre el último vecino insertado en la pila, el algoritmo se ve obligado a profundizar en una sola dirección de la cuadrícula de forma ciega, ignorando por completo la posición del nodo final y la distancia geométrica, lo que provoca que recorra esquinas enteras antes de encontrar la solución por mero descarte espacial.

4.3. Resultados

Las siguientes figuras muestran el comportamiento visual y el espectro de exploración (en color amarillo/cian) de cada solucionador.

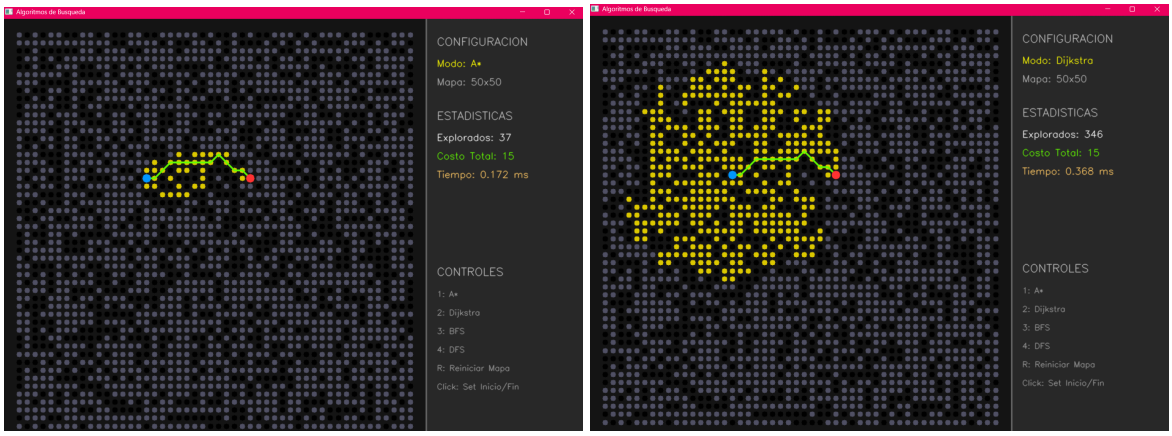

Figura 1: Izquierda: Búsqueda dirigida y minimalista usando **A***. Derecha: Exploración radial masiva en forma de burbuja expansiva generada por **Dijkstra**.

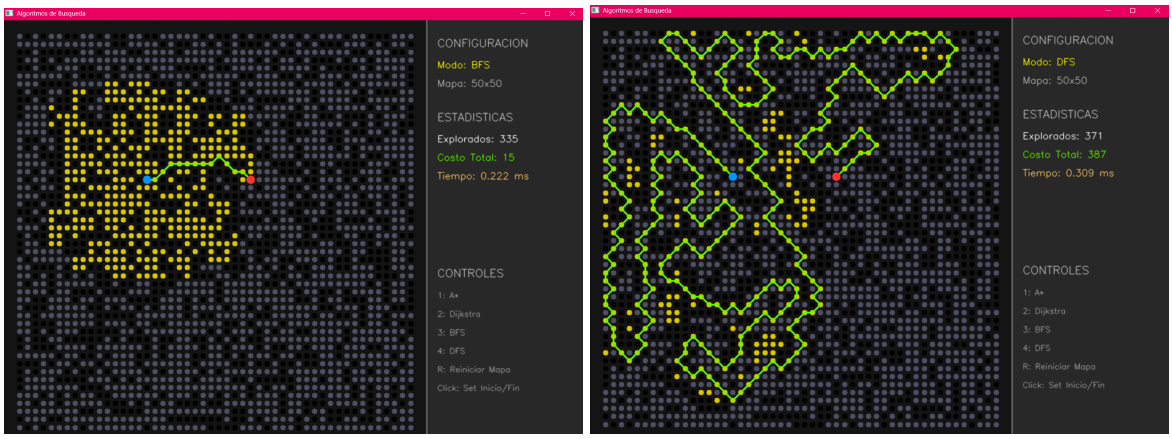

Figura 2: Izquierda: Frente de onda uniforme de **BFS**. Derecha: Trayectoria errática y caótica de **DFS** recorriendo esquinas antes de alcanzar el objetivo.

5. Conclusiones y Resultados Teóricos

- El uso de la distancia octil demostró ser matemáticamente exacto para entornos con 8 grados de libertad (permitiendo movimientos diagonales)
- A pesar de que los algoritmos no informados (**BFS** y **Dijkstra**) manejan estructuras de datos más pesadas en memoria debido a la masiva cantidad de nodos en la cola de prioridad/espera, los tiempos de ejecución medidos se mantuvieron por debajo de los 0,4 ms.
- En escenarios con obstáculos más complejos o pasajes estrechos en diagonal, **BFS** podría retornar rutas subóptimas en términos de distancia métrica, mientras que **Dijkstra** mantiene su garantía de optimalidad pase lo que pase.
- Reduce drásticamente la cantidad de nodos volcados en el reporte `nodos_explorados.txt` en comparación con Dijkstra, manteniendo idéntico el costo total métrico.
- La función de guardado automático en disco (`.txt`) facilita procesos posteriores de auditoría o analítica de datos espacial.

Anexos

El código fuente completo de este proyecto, incluyendo las librerías gráficas integradas y las instrucciones de compilación, se encuentra disponible de forma pública.

Repositorio en GitHub: `https://github.com/chuancao26/a_estrella`